

ZP-DOC-05

Data Design

Conceptual EER model, relational mapping, normalisation and the Cloudflare D1 physical schema

Version	0.9
Status	In review
Date	5 October 2026
Classification	Public
Document owner	B.M.P Banneka, Product Owner

Document control

Document information

Document ID	ZP-DOC-05
Title	Data Design
Version	0.9
Status	In review
Classification	Public
Document owner	B.M.P Banneka, Product Owner
Author	Team Kestrel (drafted with Claude, AI coding partner)
Conforms to	Enhanced ER model and mapping algorithm (Elmasri and Navathe, 7th ed.); relational normalisation to BCNF/4NF; ZP-DOC-00
Source file	docs/05-data-design/ZP-DOC-05_Data-Design.md
Repository	https://github.com/PraveenBanneka/zedpath

Revision history

Version	Date	Author	Description of change
0.9	5 Oct 2026	Team Kestrel	First issue: four EER diagrams, rule grammar, mapping, normalisation, D1 schema validated against the real UGC 2025/26 and 2024/25 data; submitted for approval
0.9	5 Oct 2026	Team Kestrel	Revised in review for CR-001 (student accounts, approved by the product owner): Section 2.5 EER model, mapping, normalisation, security conventions, 7 tables (migrations 0002 and 0003), 1:1 cardinality in the generated logical diagrams, evidence

Review and approval

This document takes effect when the approver signs below. Until then its status is as shown on the cover.

Role	Name	Signature	Date
Reviewer	Claude (AI coding partner), consistency and traceability review		
Approver	B.M.P Banneka, Product Owner		

Contents

1	Introduction	5
1.1	Purpose	5
1.2	Scope.....	5
1.3	Intended audience	5
1.4	Relationship to other documents	5
1.5	Notation	5
2	Conceptual model (EER).....	6
2.1	Offerings and cut-offs	6
2.2	Subjects and requirement rules.....	7
2.3	Provenance and trust.....	8
2.4	Journey and other routes.....	9
2.5	Student accounts (CR-001).....	10
2.6	Entity catalogue	10
3	Rule grammar.....	12
4	Mapping the EER model to relations	13
5	Normalisation.....	17
5.1	From the source form to 1NF.....	17
5.2	Functional and multi-valued dependencies.....	17
5.3	2NF, 3NF and BCNF	17
5.4	Deliberate departures	18
6	Physical design for Cloudflare D1.....	19
6.1	Conventions	19
6.2	Indexes	19
6.3	Capacity against the free plan.....	20
7	Validation against real data	21
	Appendix A: Traceability: requirements to tables	23
	Appendix B: Physical schema (migration 0001).....	24
	Appendix C: Student accounts (migration 0003).....	30
	Appendix D: Glossary.....	32
	Appendix E: References.....	33

List of figures

Figure 1: EER model of institutions, courses, offerings, intake years, selection groups and cut-offs	7
Figure 2: EER model of streams, subjects and the recursive requirement-rule tree	8
Figure 3: EER model of cited facts, sources, revisions, AI extraction, review and mistake reports	9

Figure 4: EER model of deadlines, push reminders and the specialisation of other routes	9
Figure 5: EER model of optional student accounts: sign-in, sessions, recovery and the student's results	10
Figure 6: Logical schema, admission data (generated from the migration file)	15
Figure 7: Logical schema, trust, routes and journey data (generated from the migration file)	16
Figure 8: Logical schema, optional student accounts (generated from the migration files).....	16

List of tables

Table 1: Position of this document in the ZedPath document set	5
Table 2: Entities, their type, identifier and the requirements that justify them	10
Table 3: Rule node kinds and their meaning	12
Table 4: EER-to-relational mapping steps applied.....	13
Table 5: Mapping of the student-account model	14
Table 6: Dependencies identified in R1	17
Table 7: Controlled denormalisations and why they are safe	18
Table 8: Secondary indexes and the query each serves	19
Table 9: Data volume per intake year and free-plan headroom	20
Table 10: Schema validation results (5 October 2026).....	21
Table 11: Student-account validation results (5 October 2026).....	21
Table 12: Requirements and the tables that realise them	23
Table 13: Terms used in this document.....	32
Table 14: Sources referred to in this document	33

1 Introduction

1.1 Purpose

This document designs ZedPath's data in three steps, as a database engineer would: a **conceptual** model in Enhanced Entity-Relationship (EER) notation, its **logical** relational schema derived with the standard mapping algorithm and normalised, and the **physical** schema for Cloudflare D1. The physical schema is the real migration file the application uses, and it has been proved by loading the real UGC data into it.

1.2 Scope

All data held on the server for Releases R1 and R2. By default the student's own profile lives only on their device (NFR-030). Change request CR-001 (5 October 2026) adds an **optional** student account: a student who chooses to create one keeps the same profile on the server, under a username only, so it follows them between devices and can drive reminders. Section 2.5 models it. The preference list stays on the device.

1.3 Intended audience

Developers, reviewers and anyone assessing the data design.

1.4 Relationship to other documents

Table 1: Position of this document in the ZedPath document set

Document	Relationship
ZP-DOC-03 SRS	Parent. Business rules BR-001 to BR-043 and data-quality requirements NFR-050 to NFR-052 become entities and constraints
ZP-DOC-04 Use Case Model	Parent. Nouns in the use cases name the entities
ZP-DOC-06 Software Architecture	Child. Decides how the read path uses this schema within free-plan limits
migrations/0001_initial_schema.sql	Implementation. Appendix A reproduces it verbatim
migrations/0002_reference_data.sql, 0003_student_accounts.sql	Implementation of CR-001. 0002 is generated reference data; Appendix B reproduces 0003 verbatim

1.5 Notation

Conceptual diagrams use Chen notation as extended by Elmasri and Navathe: rectangles are entities; double rectangles are **weak** entities; diamonds are relationships; double diamonds are **identifying** relationships; ovals are attributes, with keys underlined, double ovals for **multi-valued** and dashed ovals for **derived** attributes; bold lines show **total participation**; a circle marked *d* or *o* shows a **disjoint** or **overlapping specialisation**, and a circle marked *U* a **union category**. Partial keys of weak entities are named in the entity catalogue. Logical diagrams use crow's-foot (Information Engineering) notation.

2 Conceptual model (EER)

2.1 Offerings and cut-offs

The admission domain has a feature that a simple design misses: a **course of study** (for example, Management) is offered at several **institutions**; each pair is an **offering** with its own Uni-Code; an offering exists anew in each **intake year** with its own intake and requirements; and a few offerings split their seats into **selection groups** by stream or category, each with its own cut-offs. Loading the real 2025/26 cut-off table confirmed this: its 260 columns correspond to 255 Uni-Codes, because five Uni-Codes are split.

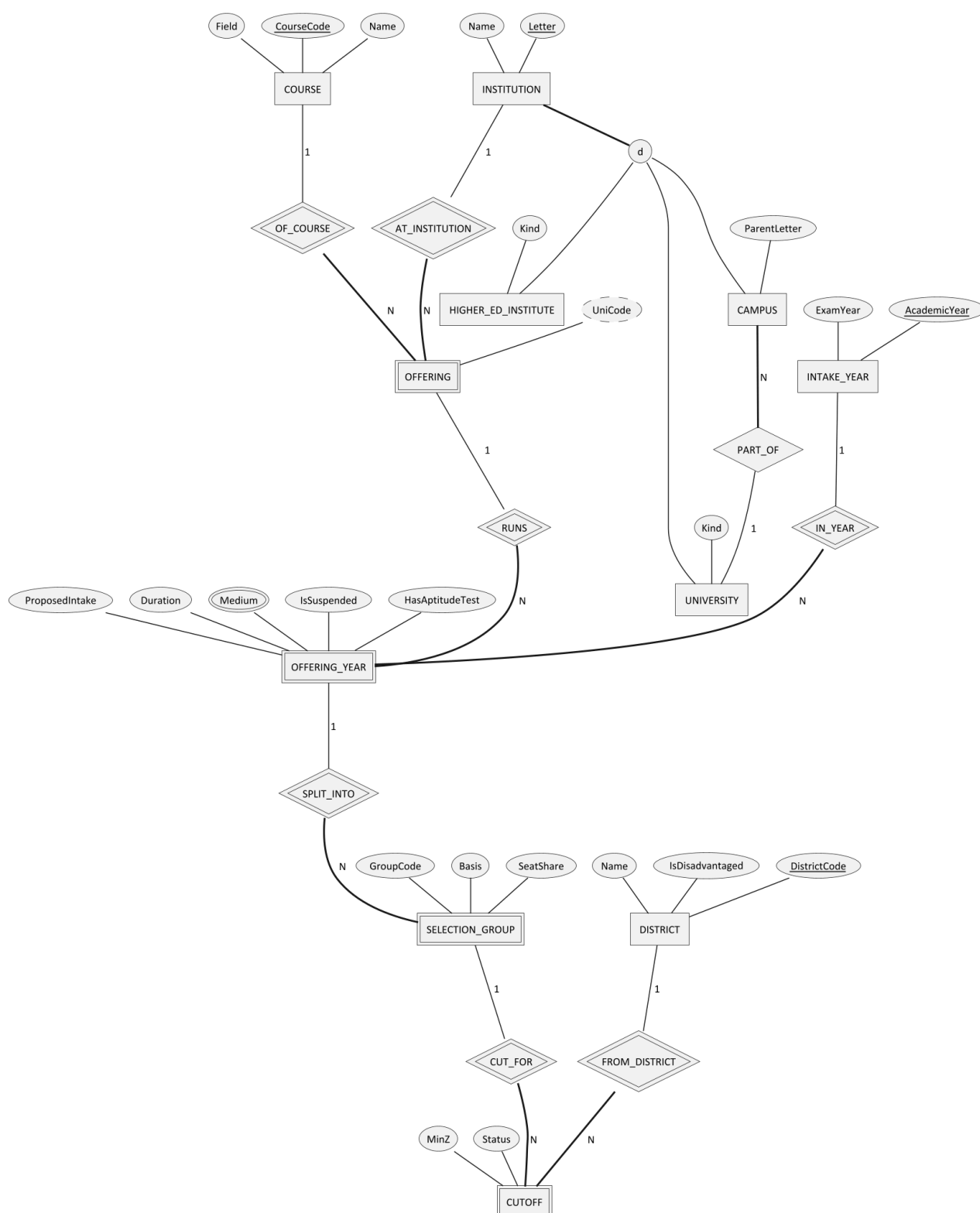


Figure 1: EER model of institutions, courses, offerings, intake years, selection groups and cut-offs

2.2 Subjects and requirement rules

Entry requirements follow twenty different patterns (BR-020). Instead of twenty columns or free text, every requirement is a **tree of rule nodes** (recursive relationship CHILD_OF) whose leaves test subjects. One structure serves state-university selection groups and other routes alike.

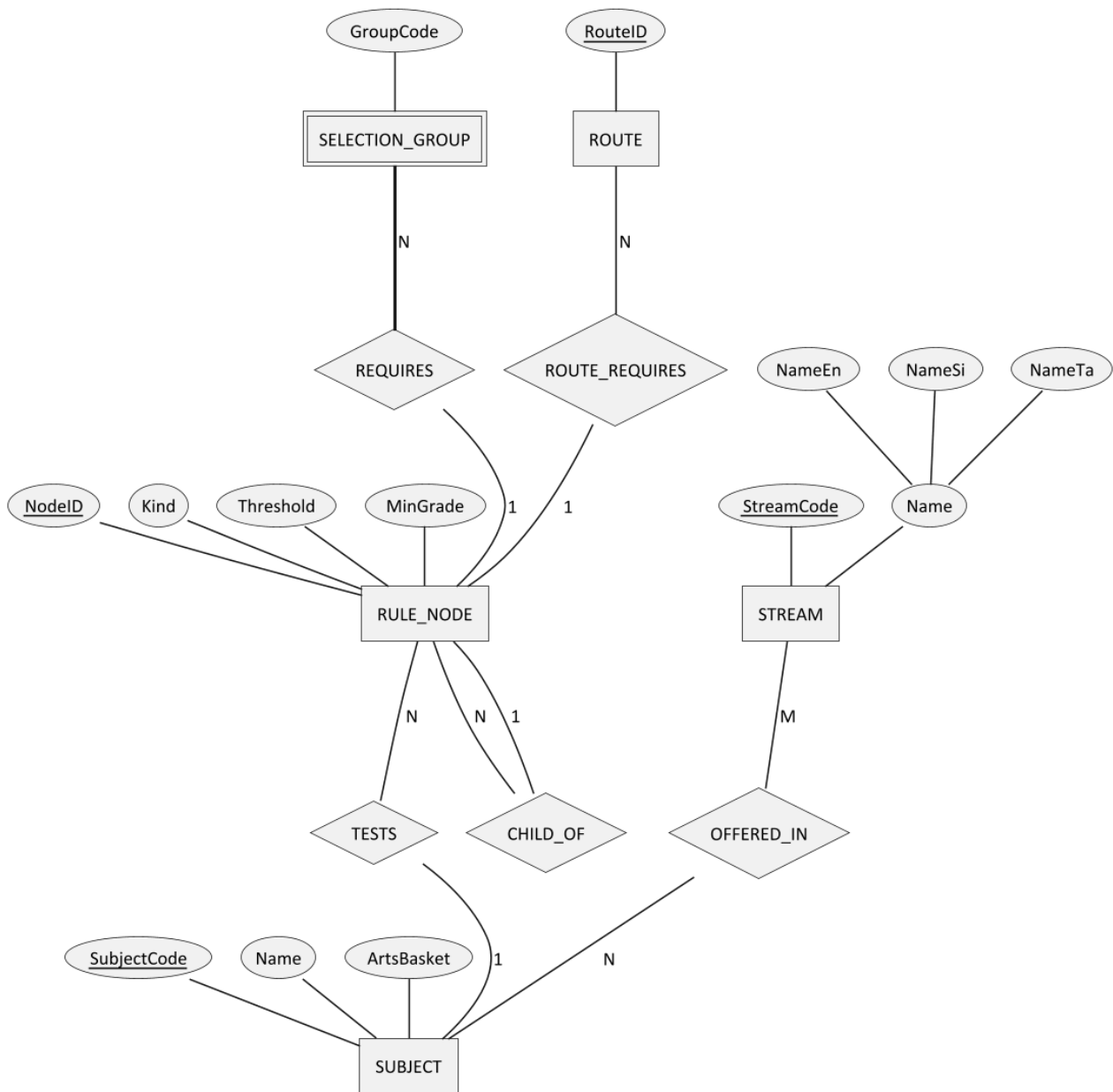


Figure 2: EER model of streams, subjects and the recursive requirement-rule tree

2.3 Provenance and trust

Every published fact must cite its source (BO-2). Cut-offs, offering details, rules, deadlines and routes are different entity types, yet each needs citations, revisions and mistake reports. The model makes them members of one **union category**, CITED_FACT, so these trust features are designed once.

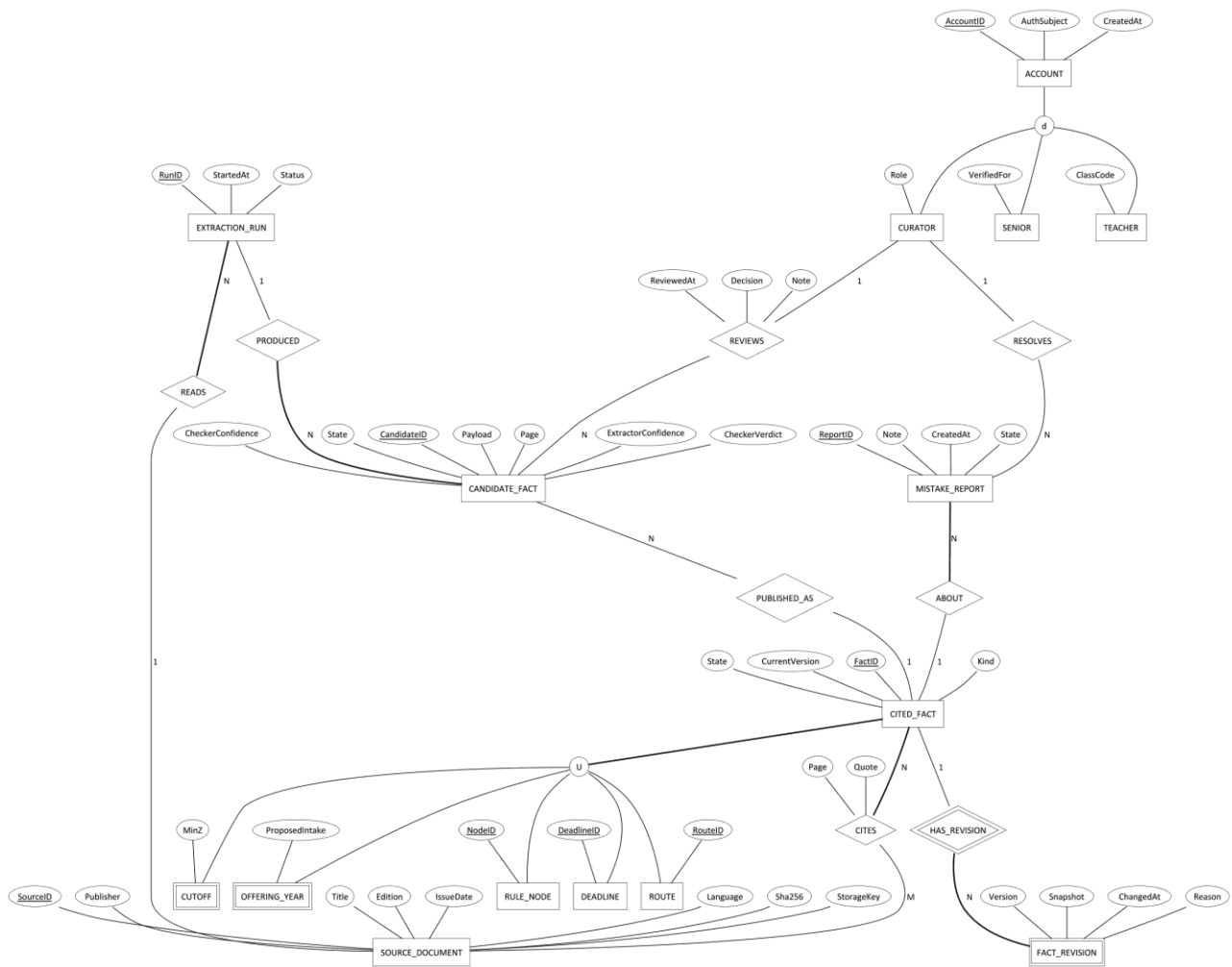


Figure 3: EER model of cited facts, sources, revisions, AI extraction, review and mistake reports

2.4 Journey and other routes

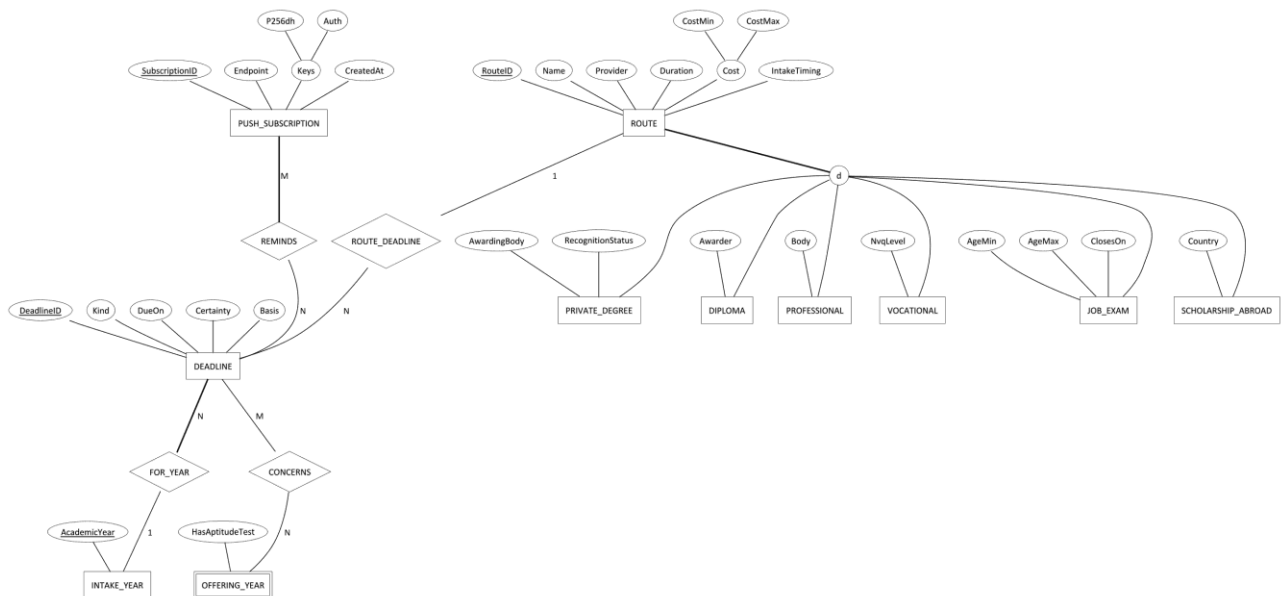


Figure 4: EER model of deadlines, push reminders and the specialisation of other routes

2.5 Student accounts (CR-001)

A student account is optional. ACCOUNT, already the superclass of the staff roles, gains a fourth subclass, STUDENT. Each way of signing in is its own weak entity of ACCOUNT, so a later identity provider (for example Google sign-in) adds an entity without touching the others. A password is never stored: PASSWORD_LOGIN holds a salt and a *verifier*, a hash of a key the student's device derives from the password. A STUDENT's results are the attributes the device already keeps: a Z-score, the stream sat (SAT_IN), the district sat from (SAT_FROM), and three subjects with grades (TOOK, whose attributes are Grade and Position). Achievements are a weak entity with partial key Position; interests are a multi-valued attribute. No name, email address or telephone number exists anywhere in the model.

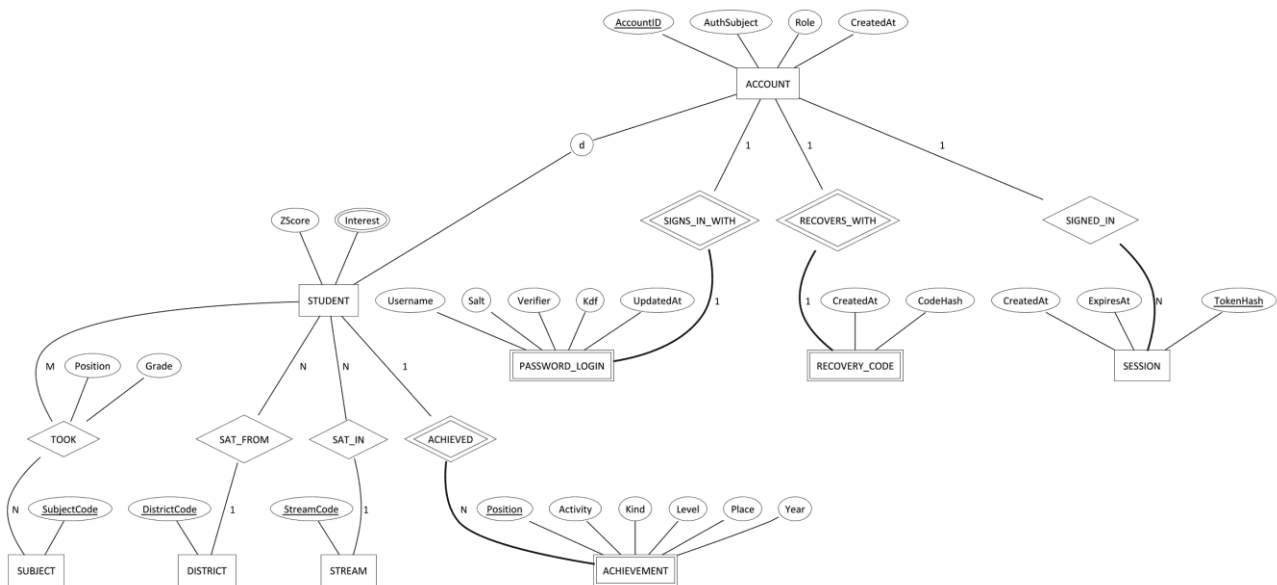


Figure 5: EER model of optional student accounts: sign-in, sessions, recovery and the student's results

2.6 Entity catalogue

Table 2: Entities, their type, identifier and the requirements that justify them

Entity	Type	Identifier	Justified by
STREAM, SUBJECT	Strong	StreamCode; SubjectCode	BR-010 to BR-012
DISTRICT	Strong	DistrictCode	BR-006, BR-007
INSTITUTION	Strong, superclass of UNIVERSITY, CAMPUS, HIGHER_ED_INSTITUTE (disjoint, total)	Letter	BR-015
COURSE	Strong	CourseCode (3 digits)	BR-015
OFFERING	Weak, owners COURSE and INSTITUTION	(CourseCode, Letter); UniCode derived	BR-015
INTAKE_YEAR	Strong	AcademicYear	BR-028
OFFERING_YEAR	Weak, owners OFFERING and INTAKE_YEAR	(UniCode, AcademicYear)	BR-013, BR-014, BR-025, BR-026
SELECTION_GROUP	Weak, owner OFFERING_YEAR	partial key GroupCode	Discovered in the 2025/26 cut-off table
CUTOFF	Weak, owners SELECTION_GROUP and DISTRICT	(group, District)	BR-028, BR-029

Entity	Type	Identifier	Justified by
RULE_NODE	Strong, recursive	NodeID	BR-020 to BR-022
CITED_FACT	Union category of CUTOFF, OFFERING_YEAR, RULE_NODE, DEADLINE, ROUTE	FactID (surrogate)	BO-2, FR-209, FR-911
SOURCE_DOCUMENT	Strong	SourceID; Sha256 unique	FR-901, FR-902
FACT_REVISION	Weak, owner CITED_FACT	partial key Version	FR-911
EXTRACTION_RUN, CANDIDATE_FACT	Strong	RunID; CandidateID	FR-903 to FR-907
ACCOUNT	Strong, superclass of CURATOR, SENIOR, TEACHER, STUDENT (disjoint, partial)	AccountID	NFR-032, FR-704, FR-706, CR-001
STUDENT	Subclass of ACCOUNT	AccountID (inherited)	CR-001
PASSWORD_LOGIN, RECOVERY_CODE	Weak, owner ACCOUNT, 1:1	AccountID (owner's key); Username unique	CR-001
SESSION	Strong, N:1 to ACCOUNT	TokenHash	CR-001
ACHIEVEMENT	Weak, owner STUDENT	partial key Position	CR-001, BR on special intakes (handbook Section 6)
MISTAKE_REPORT	Strong	ReportID	FR-910
DEADLINE	Strong	DeadlineID	FR-401 to FR-403
ROUTE	Strong, superclass of six route groups (disjoint, total)	RouteID	FR-501 to FR-507
PUSH_SUBSCRIPTION	Strong	SubscriptionID	FR-405, FR-406

3 Rule grammar

A requirement is stored as a tree of `rule_node` rows. Evaluation assigns each of the student's three A/L subjects to **at most one** leaf, which captures the handbook's meaning of phrases such as "and the third subject from".

Table 3: Rule node kinds and their meaning

Kind	Meaning	Example (handbook pattern)
ALL	Every child is satisfied, by distinct subjects	Fixed trio (pattern 1)
ANY	At least one child is satisfied	"Combined Mathematics or Physics"
AT_LEAST n	At least n children are satisfied, by distinct subjects	"two other subjects from the list" (pattern 7)
SUBJECT $s \geq g$	Leaf: the student has subject s at grade g or better, and it is not used elsewhere	"a 'C' grade in Chemistry"
ANY_SUBJECTS $n \geq g$	n further subjects of any kind, not used elsewhere	"any other two subjects" (pattern 8)
GRADE_COUNT $n \geq g$	A check on subjects already used: at least n of the listed subjects reach grade g	Medicine: two 'C' grades (pattern 2)
STREAM_IS x	Leaf: the student's stream is x	"any three subjects in the Commerce Stream" (pattern 13)
PREDICATE <i>name</i>	A named check implemented in code	ARTS_BASKETS: the Arts basket rules (pattern 17)
OL_SUBJECT $s \geq g$	Leaf in the O/L tree: the O/L subject s at grade g or better	"a Credit Pass in English" (pattern 18)

Example: Medicine (Uni-Code 001A) is ALL(SUBJECT Biology $\geq S$, SUBJECT Chemistry $\geq S$, SUBJECT Physics $\geq S$, GRADE_COUNT 2 $\geq C$ of {Biology, Chemistry, Physics}). Category splits with separate seats (pattern 14) are modelled as separate selection groups, each with its own rule tree.

4 Mapping the EER model to relations

The relational schema was derived with the standard nine-step algorithm.

Table 4: EER-to-relational mapping steps applied

Step	Construct	Applied to	Result
1	Strong entities	STREAM, SUBJECT, DISTRICT, COURSE, INTAKE_YEAR, RULE_NODE, SOURCE_DOCUMENT, EXTRACTION_RUN, CANDIDATE_FACT, MISTAKE_REPORT, DEADLINE, PUSH_SUBSCRIPTION, ACCOUNT, ROUTE	One table each, primary key from the key attribute
2	Weak entities	OFFERING; OFFERING_YEAR; SELECTION_GROUP; CUTOFF; FACT_REVISION	Owners' keys become part of the primary key: offering(course_code, institution_letter), cutoff(group_id, district_code), fact_revision(fact_id, version). OFFERING_YEAR and SELECTION_GROUP also get surrogate keys to keep later foreign keys short
3	1:1 relationships	CANDIDATE_FACT PUBLISHED_AS CITED_FACT	Foreign key published_fact_id
4	1:N relationships	PART_OF, CHILD_OF, TESTS, REQUIRES, READS, PRODUCED, ABOUT, RESOLVES, REVIEWS, FOR_YEAR	Foreign key on the N side; REVIEWS' attributes (decision, note, time) move into candidate_fact
5	M:N relationships	OFFERED_IN, CITES, CONCERNS, REMINDS	Relationship tables stream_subject, citation (with page and quote), deadline_offering_year, push_subscription_deadline
6	Multi-valued attributes	Medium of OFFERING_YEAR	Table offering_year_medium
7	N-ary relationships	None	Not needed
8	Specialisation	INSTITUTION; ACCOUNT	Option 8C, one table with a discriminator (kind, role): subclasses have almost no own attributes
8	Specialisation	ROUTE	Option 8A: superclass table plus route_private_degree and route_job_exam for subclasses with several constrained attributes; single-attribute subclasses keep the attribute on route with a guard constraint
9	Union category	CITED_FACT	Surrogate key fact_id in table fact; every member table carries a unique foreign key to it

The student-account constructs (CR-001) were mapped with the same algorithm:

Table 5: Mapping of the student-account model

Step	Construct	Result
2	Weak entities PASSWORD_LOGIN, RECOVERY_CODE (1:1 with the owner)	password_login, recovery_code, primary key = owner's account_id (at most one per account). Kept apart from account so staff rows carry no password columns and each sign-in method can be added or removed alone
2	Weak entity ACHIEVEMENT	student_achievement with surrogate achievement_id; the partial key becomes UNIQUE (account_id, position)
4	1:N SIGNED_IN	Foreign key session.account_id
4	1:N SAT_IN, SAT_FROM and the Z-score	Moved, with the Z-score, into student_profile keyed by account_id. A separate relation because participation is partial: an account may exist before results are entered, so no NULL-filled columns on account (option for partial 1:1 participation)
5	M:N TOOK (Grade, Position)	student_subject(account_id, position, subject_code, grade); at most three per student through CHECK (position 1 to 3), no subject twice through UNIQUE
6	Multi-valued Interest	student_interest(account_id, interest)
8	Specialisation STUDENT	Option 8C as before: the role discriminator gains STUDENT; triggers refuse student rows for any other role

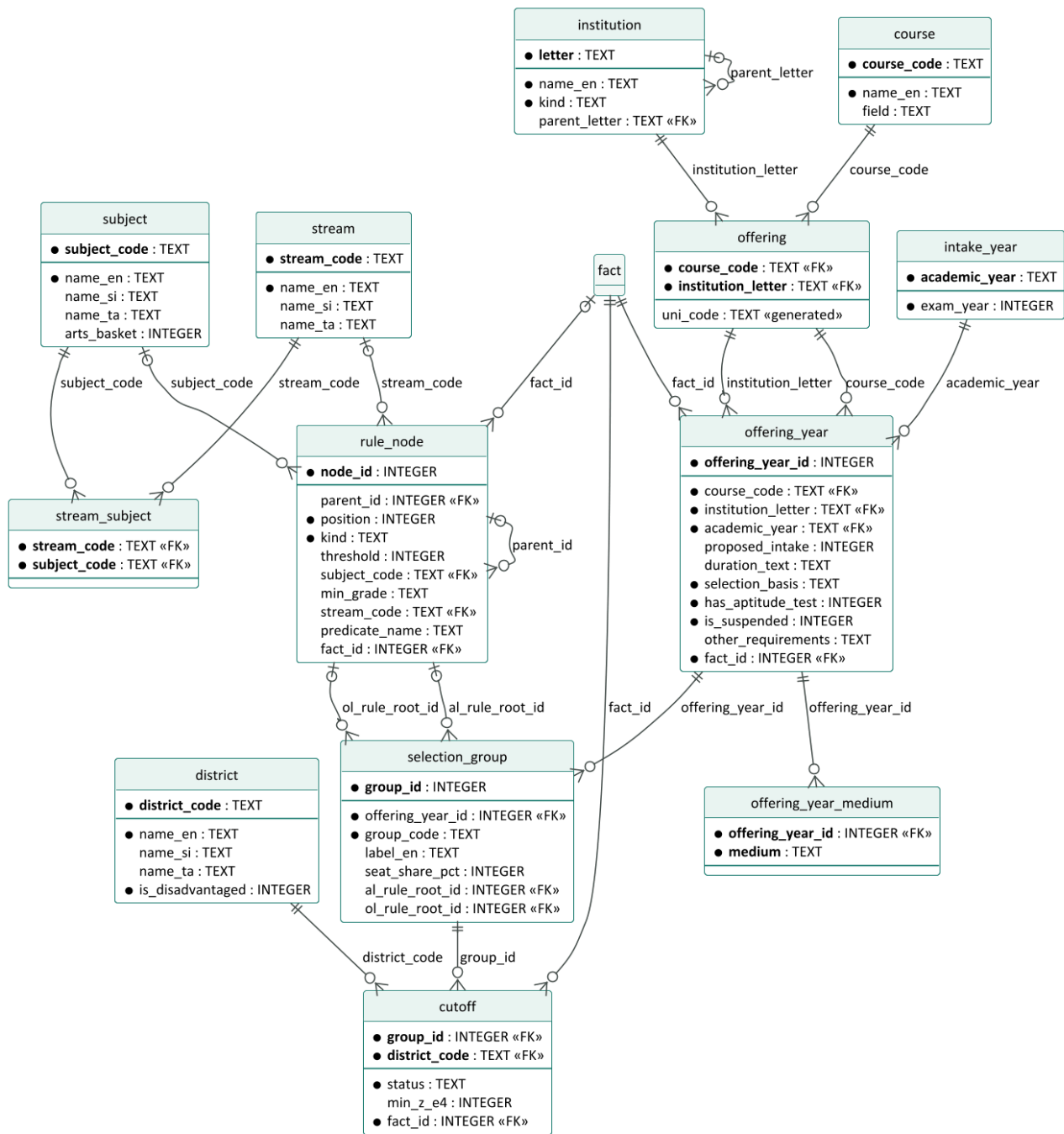


Figure 6: Logical schema, admission data (generated from the migration file)



5 Normalisation

5.1 From the source form to 1NF

The UGC cut-off table, as printed, is a single wide relation: one row per district and one column per offering, with each column header packing several facts into one string, for example "MANAGEMENT STUDIES (TV) - A [Commerce Stream] (University of Vavuniya)", plus markers * (merit only) and # (aptitude test). It violates first normal form twice: a **repeating group** (260 offering columns) and **non-atomic values** (the header). Unpivoting gives one row per (offering column, district), and splitting the header gives atomic attributes:

```
R1 ( UniCode, CourseCode, CourseName, Letter, InstitutionName, Year, GroupCode, District,
    IsDisadvantaged, MinZ, Status, ProposedIntake, SelectionBasis, HasAptitudeTest, Medium )
```

5.2 Functional and multi-valued dependencies

Table 6: Dependencies identified in R1

#	Dependency	Source of the rule
FD1	UniCode → CourseCode, Letter (and the reverse: CourseCode, Letter → UniCode)	BR-015
FD2	CourseCode → CourseName	Handbook course list
FD3	Letter → InstitutionName	Handbook Uni-Code list
FD4	District → IsDisadvantaged	BR-007
FD5	UniCode, Year → ProposedIntake, SelectionBasis, HasAptitudeTest	Handbook, per intake
FD6	UniCode, Year, GroupCode, District → MinZ, Status	Cut-off table
MVD1	UniCode, Year → Medium	Medium is independent of every other attribute

The candidate key of R1 is (UniCode, Year, GroupCode, District, Medium).

5.3 2NF, 3NF and BCNF

- **2NF** removes attributes that depend on part of the key: FD5's attributes move to OFFERING_YEAR(UniCode, Year, ...); FD4's to DISTRICT; FD1 to OFFERING. CUTOFF keeps only MinZ and Status, which need the whole key.
- **3NF** removes transitive dependencies: UniCode → CourseCode → CourseName puts CourseName in COURSE; UniCode → Letter → InstitutionName puts the name in INSTITUTION.
- **BCNF**: in every resulting relation, every determinant is a candidate key. The only relation with two candidate keys, OFFERING, has (CourseCode, Letter) and the derived UniCode, both keys, so it is in BCNF.
- **4NF**: MVD1 is non-trivial, so Medium moves to its own relation, offering_year_medium.

The same analysis applied to the provenance data separates citation (fact, source, page) from fact, because a fact may cite several pages and a page supports many facts.

The student-account relations are in BCNF by construction. In each, the only determinants are the keys: password_login has two candidate keys, account_id and username, and every other attribute depends on either. student_subject has the candidate keys (account_id, position) and (account_id, subject_code), with Grade depending on both. A student's interests are independent of their achievements (MVD AccountID → Interest), so they are kept in separate relations (4NF).

5.4 Deliberate departures

Table 7: Controlled denormalisations and why they are safe

Departure	Reason	How consistency is kept
uni_code stored alongside its parts in offering	Lookup by the code students know	Generated column, computed by the database; cannot disagree
Surrogate keys on offering_year and selection_group	Short foreign keys in the 13,000-row cutoff table	Natural keys kept as UNIQUE constraints
Read model for the student path (ZP-DOC-06)	D1 free plan allows 5 million row reads a day; a district's full eligibility view reads about 3,000 rows, which would cap ZedPath near 1,600 students a day	Built from D1 by a single job after each data change, versioned, and never edited directly

6 Physical design for Cloudflare D1

6.1 Conventions

- **STRICT tables:** every value must match its declared type.
- **Exact Z-scores:** stored as integers in ten-thousandths (`min_z_e4`; 1.4821 is stored as 14821), so comparisons are exact (NFR-051). The valid range -4.0000 to $+4.0000$ is a CHECK constraint (BR-005).
- **Enumerations** (streams, statuses, kinds, grades) are CHECK constraints, so invalid states cannot be stored.
- **Cross-column rules** are CHECK constraints: a CAMPUS must have a parent university; a VALUE cut-off must have a number and an NQC cut-off must not; an auto-published candidate must have two-model agreement at 0.90 or more (FR-905); an estimated deadline must state its basis (FR-402).
- **Foreign keys** are enforced by D1 by default (equivalent to `PRAGMA foreign_keys = ON`).
- **Personal data:** without an account, the only student-linked table is `push_subscription`, holding a browser push endpoint and keys that the student opted in to share (NFR-030). With an optional account (CR-001), the student's results, achievements and interests are kept under a username they choose; there are still no names, contact details or telephone numbers. `DELETE FROM account` cascades to every row of that student.
- **Secrets are stored only as hashes** (CR-001): `password_login.verifier` = SHA-256 of a key that the device derives with PBKDF2-HMAC-SHA256 at 600,000 rounds (the password never reaches the server); `session.token_hash` = SHA-256 of the random cookie token; `recovery_code.code_hash` = SHA-256 of an 80-bit random code. A copy of the tables therefore cannot be replayed to log in, and a guessed password costs an attacker 600,000 rounds per guess. Verifying a login costs the Worker one SHA-256, well inside the free plan's 10 ms CPU limit.

6.2 Indexes

Table 8: Secondary indexes and the query each serves

Index	Serves
<code>idx_offering_uni_code</code> (unique)	Look up an offering by Uni-Code
<code>idx_cutoff_district</code>	All cut-offs for one district: the core of eligibility and banding
<code>idx_offering_year_year</code>	All offerings of an intake year
<code>idx_rule_node_parent</code>	Loading rule trees in order
<code>idx_deadline_year_due</code>	Journey timeline by date
<code>idx_citation_source</code>	All facts citing a source (re-extraction, audits)
<code>idx_candidate_queue</code> (partial, QUEUED only)	Curator review queue
<code>idx_report_open</code> (partial, OPEN only)	Open mistake reports
<code>idx_session_account</code>	Log out on every device; clearing expired sessions at log-in
<code>idx_student_profile_stream</code> , <code>idx_student_profile_district</code>	Foreign-key checks when a stream or district row changes; counts for reminders by district
<code>idx_student_subject_subject</code>	Foreign-key checks on <code>subject</code>

6.3 Capacity against the free plan

Table 9: Data volume per intake year and free-plan headroom

Measure	Value	Free-plan limit
Cut-off rows per intake year	about 6,500	—
Database size with two intake years loaded	2.3 MB	500 MB per database
Projected size with five years and all routes	under 10 MB	500 MB per database
Rows read to rebuild the read model once	about 15,000	5 million per day
Rows written to load one intake year	about 13,500 (with facts and citations)	100,000 per day

7 Validation against real data

The design was tested by loading the official data, not sample data. `tools/seed/validate-schema.ts` applies the migration to an in-memory SQLite database (the engine D1 is built on), loads the UGC 2025/26 data and the 2024/25 cut-offs printed in the handbook, runs integrity checks, and then attempts 24 deliberately invalid inserts.

Table 10: Schema validation results (5 October 2026)

Check	Result	Expected
Uni-Codes loaded	255	255
Course codes / institutions	121 / 20	121 / 20
2025/26 proposed intake across Uni-Codes (plus 900 Arts additional intake = 42,937)	42,037	42,037
2025/26 cut-off columns matched exactly to a Uni-Code	260	260
2025/26 cut-off cells loaded / of which NQC	6,500 / 1,026	6,500 / 1,026
Uni-Codes split into more than one selection group	5	5
Uni-Codes without cut-offs	0	0
Merit-only offerings with differing district values	0	0
2024/25 cut-off cells loaded	6,425	—
Facts without a citation	0	0
Foreign-key violations	0	0
Invalid inserts rejected by constraints	24 of 24	24 of 24

The student-account tables (CR-001) are validated by `worker/account.test.ts`, which applies all three migrations to an in-memory SQLite database with foreign keys on, exactly as D1 enforces them, and drives the real API.

Table 11: Student-account validation results (5 October 2026)

Check	Result
Schema after migrations 0001 to 0003	35 tables, 190 columns, 52 foreign keys, 12 indexes, 3 triggers
Reference rows loaded by 0002 (streams / districts / subjects)	6 / 25 / 59
Invalid inserts rejected (unknown role, bad username, wrong-length hash, unknown KDF, Z-score out of range, unknown stream, district or subject, fourth subject, duplicate subject, grade D, blank or unknown achievement fields, unknown interest, expiry before creation, student rows on a staff account)	17 of 17
Deleting one account removes exactly its rows from all 8 account tables and nothing of another account	Pass
A request can read or change only its own session's account, whatever the request body says	Pass
A save that fails validation changes nothing (atomic batch)	Pass

Check	Result
Bugs planted on purpose (body-supplied account id, missing cascade, missing cross-site check, key not compared) caught by a failing test	4 of 4
Same migrations applied to the local D1 runtime (wrangler d1 migrations apply --local)	3 of 3 applied

Matching the cut-off table to Uni-Codes surfaced spelling differences between the two UGC documents ("BIO.SC" and "BIOLOGICAL SC.", "AGRI BUSINESS" and "AGRIBUSINESS", "BIO RESOURCES" and "BIORESOURCES") and one misprinted university name. The loader normalises these explicitly; no column is matched by guesswork.

Appendix A: Traceability: requirements to tables

Table 12: Requirements and the tables that realise them

Requirement	Tables
BR-005, NFR-051 (exact Z-scores)	cutoff.min_z_e4 with range CHECK
BR-006, BR-007 (districts)	district
BR-010 to BR-012 (streams, subjects)	stream, subject, stream_subject
BR-013, BR-014 (quotas, merit-only)	offering_year.selection_basis
BR-015 (Uni-Codes)	course, institution, offering
BR-020 to BR-024 (requirements)	rule_node, selection_group.al_rule_root_id, ol_rule_root_id, offering_year.other_requirements
BR-025, BR-026 (aptitude, suspended)	offering_year.has_aptitude_test, is_suspended
BR-028, BR-029 (cut-offs, NQC)	selection_group, cutoff
FR-209, BO-2, NFR-050 (every fact sourced)	fact, citation, source_document
FR-401 to FR-406 (journey, reminders)	deadline, deadline_offering_year, push_subscription, push_subscription_deadline
FR-501 to FR-507 (other routes)	route, route_private_degree, route_job_exam
FR-901 to FR-911 (data and trust)	source_document, extraction_run, candidate_fact, fact_revision, mistake_report, account
CR-001 (optional student account, cross-device profile)	account (role STUDENT), password_login, session, recovery_code, student_profile, student_subject, student_achievement, student_interest

Appendix B: Physical schema (migration 0001)

Reproduced verbatim from `migrations/0001_initial_schema.sql`.

```
-- ZedPath D1 schema, migration 0001.
-- Designed in ZP-DOC-05 Data Design (conceptual EER -> relational mapping -> normalisation to
BCNF/4NF).
-- Conventions: STRICT tables; snake_case; exact Z-scores as INTEGER ten-thousandths (z_e4 = Z x
10000,
-- e.g. 1.4821 -> 14821) so comparisons are exact (NFR-051); timestamps as ISO 8601 UTC text;
-- every published fact has a row in `fact` and at least one `citation` (BO-2, NFR-050).
-- D1 enforces foreign keys by default (equivalent to PRAGMA foreign_keys = ON).

-- ----- provenance (needed first: other
tables reference fact)

CREATE TABLE fact (
  fact_id      INTEGER PRIMARY KEY,
  kind         TEXT      NOT NULL CHECK (kind IN
('CUTOFF', 'OFFERING_YEAR', 'RULE', 'DEADLINE', 'ROUTE')),
  state        TEXT      NOT NULL DEFAULT 'PUBLISHED' CHECK (state IN ('PUBLISHED', 'WITHDRAWN')),
  current_version INTEGER NOT NULL DEFAULT 1 CHECK (current_version >= 1),
  updated_at   TEXT      NOT NULL CHECK (updated_at GLOB '[0-9][0-9][0-9][0-9]-[0-9][0-9]-[0-9][0-9]T*Z')
) STRICT;

CREATE TABLE source_document (
  source_id    INTEGER PRIMARY KEY,
  publisher    TEXT NOT NULL,
  title        TEXT NOT NULL,
  edition      TEXT NOT NULL,
  issue_date   TEXT CHECK (issue_date IS NULL OR issue_date GLOB '[0-9][0-9][0-9][0-9]-[0-9][0-9]-[0-9][0-9]'),
  language     TEXT NOT NULL CHECK (language IN ('EN', 'SI', 'TA')),
  sha256       TEXT NOT NULL UNIQUE CHECK (length(sha256) = 64 AND sha256 NOT GLOB '*[^0-9a-f]*'),
  storage_key  TEXT NOT NULL UNIQUE,
  redistributed -- R2 object key; the file itself is never
  url          TEXT,
  retrieved_at TEXT NOT NULL
) STRICT;

CREATE TABLE citation (
  fact_id      INTEGER NOT NULL REFERENCES fact(fact_id) ON DELETE CASCADE,
  source_id    INTEGER NOT NULL REFERENCES source_document(source_id),
  page         INTEGER NOT NULL CHECK (page >= 1),
  quote        TEXT,
  PRIMARY KEY (fact_id, source_id, page)
) STRICT;
CREATE INDEX idx_citation_source ON citation(source_id);

CREATE TABLE account (
  account_id    INTEGER PRIMARY KEY,
  auth_subject  TEXT NOT NULL UNIQUE,
  role          TEXT NOT NULL CHECK (role IN ('CURATOR', 'SENIOR', 'TEACHER')),
  verified_for_letter TEXT REFERENCES institution(letter),
  class_code    TEXT UNIQUE,
  created_at    TEXT NOT NULL,
  CHECK ((role = 'SENIOR') = (verified_for_letter IS NOT NULL)),
  CHECK ((role = 'TEACHER') = (class_code IS NOT NULL))
) STRICT;

CREATE TABLE fact_revision (
  fact_id      INTEGER NOT NULL REFERENCES fact(fact_id) ON DELETE CASCADE,
  version      INTEGER NOT NULL CHECK (version >= 1),
  snapshot_json TEXT      NOT NULL CHECK (json_valid(snapshot_json)),
  changed_at   TEXT      NOT NULL,
  changed_by   INTEGER REFERENCES account(account_id),
```

```

    reason          TEXT    NOT NULL,
    PRIMARY KEY (fact_id, version)
) STRICT;

----- reference data

CREATE TABLE stream (
    stream_code TEXT PRIMARY KEY CHECK (stream_code IN ('ARTS','COMMERCE','BIO','PHYS','ET','BST')),
    name_en     TEXT NOT NULL,
    name_si     TEXT,
    name_ta     TEXT
) STRICT;

CREATE TABLE subject (
    subject_code TEXT PRIMARY KEY CHECK (length(subject_code) BETWEEN 2 AND 12 AND subject_code NOT
GLOB '*[^A-Z0-9_]*'),
    name_en     TEXT NOT NULL UNIQUE,
    name_si     TEXT,
    name_ta     TEXT,
    arts_basket INTEGER CHECK (arts_basket IS NULL OR arts_basket BETWEEN 1 AND 4)
) STRICT;

CREATE TABLE stream_subject (
    stream_code TEXT NOT NULL REFERENCES stream(stream_code),
    subject_code TEXT NOT NULL REFERENCES subject(subject_code),
    PRIMARY KEY (stream_code, subject_code)
) STRICT, WITHOUT ROWID;

CREATE TABLE district (
    district_code TEXT PRIMARY KEY CHECK (length(district_code) = 3 AND district_code NOT GLOB
'*[^A-Z]*'),
    name_en     TEXT NOT NULL UNIQUE,
    name_si     TEXT,
    name_ta     TEXT,
    is_disadvantaged INTEGER NOT NULL CHECK (is_disadvantaged IN (0,1))    -- BR-007
) STRICT;

-- Specialisation UNIVERSITY / CAMPUS / HIGHER_ED_INSTITUTE mapped to one table with a discriminator
-- (Elmasri and Navathe option 8C): the subclasses have almost no own attributes.
CREATE TABLE institution (
    letter        TEXT PRIMARY KEY CHECK (length(letter) = 1 AND letter GLOB '[A-Z]'),
    name_en       TEXT NOT NULL UNIQUE,
    kind          TEXT NOT NULL CHECK (kind IN ('UNIVERSITY','CAMPUS','HEI')),
    parent_letter TEXT REFERENCES institution(letter),
    CHECK ((kind = 'CAMPUS') = (parent_letter IS NOT NULL))
) STRICT;

CREATE TABLE course (
    course_code TEXT PRIMARY KEY CHECK (course_code GLOB '[0-9][0-9][0-9]'),
    name_en     TEXT NOT NULL,
    field       TEXT
) STRICT;

-- Weak entity OFFERING, identified by COURSE and INSTITUTION; the Uni-Code is derived (BR-015).
CREATE TABLE offering (
    course_code      TEXT NOT NULL REFERENCES course(course_code),
    institution_letter TEXT NOT NULL REFERENCES institution(letter),
    uni_code         TEXT GENERATED ALWAYS AS (course_code || institution_letter) STORED,
    PRIMARY KEY (course_code, institution_letter)
) STRICT;
CREATE UNIQUE INDEX idx_offering_uni_code ON offering(uni_code);

CREATE TABLE intake_year (
    academic_year TEXT PRIMARY KEY CHECK (academic_year GLOB '[0-9][0-9][0-9][0-9]/[0-9][0-9][0-9][0-9]'),
    exam_year     INTEGER NOT NULL,
    CHECK (exam_year = CAST(substr(academic_year, 1, 4) AS INTEGER)),
    CHECK (CAST(substr(academic_year, 6, 4) AS INTEGER) = exam_year + 1)
) STRICT;

```

```

----- requirement rules (recursive
rule tree)
-- Kinds: ALL / ANY (boolean), AT_LEAST n of the children, SUBJECT (leaf: subject at min_grade),
-- ANY_SUBJECTS n (n further subjects of any kind), GRADE_COUNT (at least n of the child subjects at
min_grade,
-- a check on subjects already used), STREAM_IS (leaf), PREDICATE (named check implemented in code,
e.g. ARTS_BASKETS),
-- OL_SUBJECT (leaf: G.C.E. O/L subject at min_grade). Semantics in ZP-DOC-05 Section "Rule
grammar".
CREATE TABLE rule_node (
  node_id          INTEGER PRIMARY KEY,
  parent_id        INTEGER REFERENCES rule_node(node_id) ON DELETE CASCADE,
  position         INTEGER NOT NULL DEFAULT 0 CHECK (position >= 0),
  kind             TEXT NOT NULL CHECK (kind IN
('ALL', 'ANY', 'AT_LEAST', 'SUBJECT', 'ANY_SUBJECTS', 'GRADE_COUNT',
  'STREAM_IS', 'PREDICATE', 'OL_SUBJECT')),
  threshold        INTEGER CHECK (threshold IS NULL OR threshold BETWEEN 1 AND 3),
  subject_code     TEXT REFERENCES subject(subject_code),
  min_grade        TEXT CHECK (min_grade IS NULL OR min_grade IN ('A', 'B', 'C', 'S')),
  stream_code      TEXT REFERENCES stream(stream_code),
  predicate_name   TEXT CHECK (predicate_name IS NULL OR predicate_name IN ('ARTS_BASKETS')),
  fact_id          INTEGER UNIQUE REFERENCES fact(fact_id), -- set on root nodes only: the rule is
the cited fact
  CHECK (kind NOT IN ('SUBJECT', 'OL_SUBJECT') OR (subject_code IS NOT NULL AND min_grade IS NOT
NULL)),
  CHECK (kind NOT IN ('AT_LEAST', 'ANY_SUBJECTS', 'GRADE_COUNT') OR threshold IS NOT NULL),
  CHECK (kind <> 'GRADE_COUNT' OR min_grade IS NOT NULL),
  CHECK (kind <> 'ANY_SUBJECTS' OR min_grade IS NOT NULL),
  CHECK ((kind = 'STREAM_IS') = (stream_code IS NOT NULL)),
  CHECK ((kind = 'PREDICATE') = (predicate_name IS NOT NULL)),
  CHECK (subject_code IS NULL OR kind IN ('SUBJECT', 'OL_SUBJECT')),
  CHECK (fact_id IS NULL OR parent_id IS NULL)
) STRICT;
CREATE INDEX idx_rule_node_parent ON rule_node(parent_id, position);

----- offerings per year, selection
groups, cut-offs

CREATE TABLE offering_year (
  offering_year_id  INTEGER PRIMARY KEY,
  course_code       TEXT NOT NULL,
  institution_letter TEXT NOT NULL,
  academic_year     TEXT NOT NULL REFERENCES intake_year(academic_year),
  proposed_intake   INTEGER CHECK (proposed_intake IS NULL OR proposed_intake >= 0),
  duration_text     TEXT,
  selection_basis   TEXT NOT NULL CHECK (selection_basis IN ('QUOTA', 'MERIT_ONLY')), -- BR-
013, BR-014
  has_aptitude_test INTEGER NOT NULL CHECK (has_aptitude_test IN (0,1)), -- BR-025
  is_suspended     INTEGER NOT NULL DEFAULT 0 CHECK (is_suspended IN (0,1)), -- BR-026
  other_requirements TEXT, -- BR-023
  (not evaluated)
  fact_id           INTEGER NOT NULL UNIQUE REFERENCES fact(fact_id),
  FOREIGN KEY (course_code, institution_letter) REFERENCES offering(course_code,
institution_letter),
  UNIQUE (course_code, institution_letter, academic_year)
) STRICT;
CREATE INDEX idx_offering_year_year ON offering_year(academic_year);

-- Multi-valued attribute Medium (4NF: independent of every other offering-year attribute).
CREATE TABLE offering_year_medium (
  offering_year_id  INTEGER NOT NULL REFERENCES offering_year(offering_year_id) ON DELETE CASCADE,
  medium            TEXT NOT NULL CHECK (medium IN ('SI', 'TA', 'EN')),
  PRIMARY KEY (offering_year_id, medium)
) STRICT, WITHOUT ROWID;

-- Weak entity SELECTION_GROUP: most offerings have one group 'ALL'; a few split their seats by
stream or category
-- (e.g. Food Business Management: Biological/Physical Science vs Commerce), each with its own rule
and cut-offs.

```

```

CREATE TABLE selection_group (
  group_id          INTEGER PRIMARY KEY,
  offering_year_id  INTEGER NOT NULL REFERENCES offering_year(offering_year_id) ON DELETE CASCADE,
  group_code        TEXT    NOT NULL DEFAULT 'ALL' CHECK (length(group_code) BETWEEN 1 AND 16),
  label_en          TEXT,
  seat_share_pct    INTEGER CHECK (seat_share_pct IS NULL OR seat_share_pct BETWEEN 1 AND 100),
  al_rule_root_id   INTEGER REFERENCES rule_node(node_id),
  ol_rule_root_id   INTEGER REFERENCES rule_node(node_id),
  UNIQUE (offering_year_id, group_code)
) STRICT;

-- Weak entity CUTOFF, identified by SELECTION_GROUP and DISTRICT (BR-028, BR-029).
CREATE TABLE cutoff (
  group_id          INTEGER NOT NULL REFERENCES selection_group(group_id) ON DELETE CASCADE,
  district_code     TEXT    NOT NULL REFERENCES district(district_code),
  status            TEXT    NOT NULL CHECK (status IN ('VALUE', 'NQC')),
  min_z_e4          INTEGER CHECK (min_z_e4 IS NULL OR min_z_e4 BETWEEN -40000 AND 40000),
  fact_id           INTEGER NOT NULL UNIQUE REFERENCES fact(fact_id),
  CHECK ((status = 'VALUE') = (min_z_e4 IS NOT NULL)),
  PRIMARY KEY (group_id, district_code)
) STRICT;
CREATE INDEX idx_cutoff_district ON cutoff(district_code, group_id);

-- ----- other routes (specialisation
ROUTE)
-- Superclass table + subclass tables for the subclasses with their own constrained attributes
-- (option 8A); single-attribute subclasses keep that attribute on the superclass with a guard
CHECK.

CREATE TABLE route (
  route_id          INTEGER PRIMARY KEY,
  route_group       TEXT NOT NULL CHECK (route_group IN
('PRIVATE_DEGREE', 'DIPLOMA', 'PROFESSIONAL', 'VOCATIONAL',
'JOB_EXAM', 'SCHOLARSHIP_ABROAD')),
  name_en           TEXT NOT NULL,
  provider          TEXT NOT NULL,
  duration_text     TEXT,
  cost_min_lkr      INTEGER CHECK (cost_min_lkr IS NULL OR cost_min_lkr >= 0),
  cost_max_lkr      INTEGER CHECK (cost_max_lkr IS NULL OR cost_max_lkr >= 0),
  intake_timing     TEXT,
  nvq_level         INTEGER CHECK (nvq_level IS NULL OR (route_group = 'VOCATIONAL' AND nvq_level
BETWEEN 1 AND 7)),
  country           TEXT    CHECK (country IS NULL OR route_group = 'SCHOLARSHIP_ABROAD'),
  rule_root_id      INTEGER REFERENCES rule_node(node_id),
  fact_id           INTEGER NOT NULL UNIQUE REFERENCES fact(fact_id),
  CHECK (cost_min_lkr IS NULL OR cost_max_lkr IS NULL OR cost_min_lkr <= cost_max_lkr)
) STRICT;

CREATE TABLE route_private_degree (
  route_id          INTEGER PRIMARY KEY REFERENCES route(route_id) ON DELETE CASCADE,
  awarding_body     TEXT NOT NULL,
  recognition_status TEXT NOT NULL CHECK (recognition_status IN ('LISTED', 'NOT_LISTED', 'UNKNOWN'))
) STRICT;

CREATE TABLE route_job_exam (
  route_id          INTEGER PRIMARY KEY REFERENCES route(route_id) ON DELETE CASCADE,
  age_min           INTEGER CHECK (age_min IS NULL OR age_min BETWEEN 14 AND 60),
  age_max           INTEGER CHECK (age_max IS NULL OR age_max BETWEEN 14 AND 60),
  closes_on         TEXT CHECK (closes_on IS NULL OR closes_on GLOB '[0-9][0-9][0-9][0-9]-[0-9][0-9]-[0-
9][0-9]'),
  CHECK (age_min IS NULL OR age_max IS NULL OR age_min <= age_max)
) STRICT;

-- ----- journey: deadlines and reminders

CREATE TABLE deadline (
  deadline_id       INTEGER PRIMARY KEY,
  academic_year     TEXT NOT NULL REFERENCES intake_year(academic_year),
  kind              TEXT NOT NULL CHECK (kind IN
('UGC_APPLICATION_OPEN', 'UGC_APPLICATION_CLOSE', 'PREFERENCE_CHANGE_CLOSE',

```

```

'APTITUDE_APPLICATION','APTITUDE_TEST','CUTOFF_RELEASE','APPEAL_CLOSE',
'REGISTRATION_CLOSE','ROUTE_INTAKE','JOB_EXAM_CLOSE','OTHER')),
  title_en      TEXT NOT NULL,
  due_on        TEXT NOT NULL CHECK (due_on GLOB '[0-9][0-9][0-9][0-9]-[0-9][0-9]-[0-9][0-9]'),
  certainty     TEXT NOT NULL CHECK (certainty IN ('CONFIRMED','ESTIMATED')),
/ FR-402
  basis_text    TEXT,
  route_id      INTEGER REFERENCES route(route_id),
  fact_id       INTEGER NOT NULL UNIQUE REFERENCES fact(fact_id),
  CHECK (certainty = 'CONFIRMED' OR basis_text IS NOT NULL)
) STRICT;
CREATE INDEX idx_deadline_year_due ON deadline(academic_year, due_on);

CREATE TABLE deadline_offering_year (
  deadline_id   INTEGER NOT NULL REFERENCES deadline(deadline_id) ON DELETE CASCADE,
  offering_year_id INTEGER NOT NULL REFERENCES offering_year(offering_year_id) ON DELETE CASCADE,
  PRIMARY KEY (deadline_id, offering_year_id)
) STRICT, WITHOUT ROWID;

-- Only data a student explicitly opted in to (NFR-030): a push endpoint, never a name or contact
-- detail.
CREATE TABLE push_subscription (
  subscription_id TEXT PRIMARY KEY CHECK (length(subscription_id) = 32),
  endpoint        TEXT NOT NULL UNIQUE CHECK (endpoint GLOB 'https://*'),
  p256dh         TEXT NOT NULL,
  auth           TEXT NOT NULL,
  created_at     TEXT NOT NULL
) STRICT;

CREATE TABLE push_subscription_deadline (
  subscription_id TEXT NOT NULL REFERENCES push_subscription(subscription_id) ON DELETE CASCADE,
  deadline_id     INTEGER NOT NULL REFERENCES deadline(deadline_id) ON DELETE CASCADE,
  PRIMARY KEY (subscription_id, deadline_id)
) STRICT, WITHOUT ROWID;

----- extraction, review and reports

CREATE TABLE extraction_run (
  run_id        INTEGER PRIMARY KEY,
  source_id     INTEGER NOT NULL REFERENCES source_document(source_id),
  started_at    TEXT NOT NULL,
  finished_at   TEXT,
  status        TEXT NOT NULL CHECK (status IN ('RUNNING','SUCCEEDED','FAILED','PAUSED')),
  n_published   INTEGER NOT NULL DEFAULT 0 CHECK (n_published >= 0),
  n_queued      INTEGER NOT NULL DEFAULT 0 CHECK (n_queued >= 0),
  n_rejected    INTEGER NOT NULL DEFAULT 0 CHECK (n_rejected >= 0),
  CHECK ((status = 'RUNNING') = (finished_at IS NULL))
) STRICT;

-- REVIEWS (1:N, with attributes) is mapped into candidate_fact with the foreign-key approach
-- (mapping step 4).
CREATE TABLE candidate_fact (
  candidate_id  INTEGER PRIMARY KEY,
  run_id        INTEGER NOT NULL REFERENCES extraction_run(run_id) ON DELETE CASCADE,
  kind          TEXT NOT NULL CHECK (kind IN
('CUTOFF','OFFERING_YEAR','RULE','DEADLINE','ROUTE')),
  payload_json  TEXT NOT NULL CHECK (json_valid(payload_json)),
  page         INTEGER NOT NULL CHECK (page >= 1),
  extractor_confidence REAL NOT NULL CHECK (extractor_confidence BETWEEN 0 AND 1),
  checker_verdict TEXT CHECK (checker_verdict IS NULL OR checker_verdict IN
('AGREE','DISAGREE')),
  checker_confidence REAL CHECK (checker_confidence IS NULL OR checker_confidence BETWEEN 0
AND 1),
  state        TEXT NOT NULL CHECK (state IN
('AUTO_PUBLISHED','QUEUED','APPROVED','CORRECTED','REJECTED')),
  published_fact_id INTEGER REFERENCES fact(fact_id),
  reviewer_id   INTEGER REFERENCES account(account_id),
  review_note   TEXT,

```

```
    reviewed_at      TEXT,
    CHECK ((state IN ('AUTO_PUBLISHED','APPROVED','CORRECTED')) = (published_fact_id IS NOT NULL)),
    CHECK ((state IN ('APPROVED','CORRECTED','REJECTED')) = (reviewer_id IS NOT NULL AND reviewed_at
IS NOT NULL)),
    CHECK (state <> 'AUTO_PUBLISHED' OR (checker_verdict = 'AGREE' AND extractor_confidence >= 0.90
                                AND checker_confidence >= 0.90))                -- FR-
905
) STRICT;
CREATE INDEX idx_candidate_queue ON candidate_fact(state) WHERE state = 'QUEUED';

CREATE TABLE mistake_report (
  report_id      INTEGER PRIMARY KEY,
  fact_id        INTEGER NOT NULL REFERENCES fact(fact_id),
  note           TEXT CHECK (note IS NULL OR length(note) <= 1000),
  created_at     TEXT NOT NULL,
  state          TEXT NOT NULL DEFAULT 'OPEN' CHECK (state IN ('OPEN','FIXED','CLOSED')),
  resolved_by    INTEGER REFERENCES account(account_id),
  resolution     TEXT,
  CHECK ((state = 'OPEN') = (resolved_by IS NULL))
) STRICT;
CREATE INDEX idx_report_open ON mistake_report(state) WHERE state = 'OPEN';
```

Appendix C: Student accounts (migration 0003)

Reproduced verbatim from `migrations/0003_student_accounts.sql`. Migration 0002 inserts the reference data (6 streams, 25 districts, 59 subjects) and is generated by `tools/seed/reference-sql.ts` from the compiled rulebook.

```
-- ZedPath D1 migration 0003: student accounts (CR-001, approved by Praveen 2026-10-05 "oo
accounts").
-- Design: ZP-DOC-05 Section "Student accounts". One identity table (account) for every role; each
way of signing in
-- is its own table hanging off it (password_login now, an identity provider such as Google later).
-- Privacy (NFR-030): a username only. No name, email or phone is ever stored. Deleting an account
deletes every row
-- below it (ON DELETE CASCADE), enforced by D1, which always enforces foreign keys.

-- account: add the STUDENT role. SQLite cannot change a CHECK in place, so the table is rebuilt. It
holds no rows
-- yet in any environment (no staff accounts exist), so the copy is a no-op kept for correctness.
CREATE TABLE account_v2 (
  account_id      INTEGER PRIMARY KEY,
  auth_subject    TEXT NOT NULL UNIQUE,          -- opaque: 'local:<32 hex>' for password
logins, provider subject later
  role            TEXT NOT NULL CHECK (role IN ('CURATOR','SENIOR','TEACHER','STUDENT')),
  verified_for_letter TEXT REFERENCES institution(letter),
  class_code      TEXT UNIQUE,
  created_at      TEXT NOT NULL,
  CHECK ((role = 'SENIOR') = (verified_for_letter IS NOT NULL)),
  CHECK ((role = 'TEACHER') = (class_code IS NOT NULL))
) STRICT;
INSERT INTO account_v2 SELECT account_id, auth_subject, role, verified_for_letter, class_code,
created_at FROM account;
DROP TABLE account;
ALTER TABLE account_v2 RENAME TO account;

-- Username + password. The phone derives key = PBKDF2-SHA256(password, salt, 60000) and sends only
the key;
-- the server stores verifier = SHA-256(key). Salt = HMAC(PEPPER, username), stored so a PEPPER
change never locks
-- anyone out; unknown usernames get the same HMAC, so the salt endpoint does not reveal which
usernames exist.
CREATE TABLE password_login (
  account_id      INTEGER PRIMARY KEY REFERENCES account(account_id) ON DELETE CASCADE,
  username        TEXT NOT NULL UNIQUE CHECK (length(username) BETWEEN 3 AND 24 AND username NOT GLOB
'*[^a-z0-9_]*'),
  salt            TEXT NOT NULL CHECK (length(salt) = 32 AND salt NOT GLOB '*[^0-9a-f]*'),
  verifier        TEXT NOT NULL CHECK (length(verifier) = 64 AND verifier NOT GLOB '*[^0-9a-f]*'),
  kdf             TEXT NOT NULL CHECK (kdf IN ('PBKDF2-SHA256-60000')),
  updated_at      TEXT NOT NULL
) STRICT;

-- Sessions: the cookie holds a random token; only its SHA-256 is stored, so a leaked table cannot
be replayed.
CREATE TABLE session (
  token_hash      TEXT PRIMARY KEY CHECK (length(token_hash) = 64 AND token_hash NOT GLOB '*[^0-9a-
f]*'),
  account_id      INTEGER NOT NULL REFERENCES account(account_id) ON DELETE CASCADE,
  created_at      TEXT NOT NULL,
  expires_at      TEXT NOT NULL,
  CHECK (expires_at > created_at)
) STRICT, WITHOUT ROWID;
CREATE INDEX idx_session_account ON session(account_id);

-- Recovery codes replace "forgot password" emails (the Free plan cannot send email). 80 random
bits, so a plain
-- SHA-256 is enough; one active code per account, replaced on every use.
CREATE TABLE recovery_code (
  account_id      INTEGER PRIMARY KEY REFERENCES account(account_id) ON DELETE CASCADE,
```

```

code_hash TEXT NOT NULL CHECK (length(code_hash) = 64 AND code_hash NOT GLOB '*[^\0-9a-f]*'),
created_at TEXT NOT NULL
) STRICT;

-- The student's results: the same facts the phone keeps (FR-101..105), now kept for them across
devices.
CREATE TABLE student_profile (
  account_id INTEGER PRIMARY KEY REFERENCES account(account_id) ON DELETE CASCADE,
  stream_code TEXT NOT NULL REFERENCES stream(stream_code),
  district_code TEXT NOT NULL REFERENCES district(district_code),
  z_e4 INTEGER NOT NULL CHECK (z_e4 BETWEEN -40000 AND 40000),
  updated_at TEXT NOT NULL
) STRICT;
CREATE INDEX idx_student_profile_stream ON student_profile(stream_code);
CREATE INDEX idx_student_profile_district ON student_profile(district_code);

-- Exactly three subjects: position 1..3 caps it at three in the database; the API refuses fewer.
CREATE TABLE student_subject (
  account_id INTEGER NOT NULL REFERENCES student_profile(account_id) ON DELETE CASCADE,
  position INTEGER NOT NULL CHECK (position BETWEEN 1 AND 3),
  subject_code TEXT NOT NULL REFERENCES subject(subject_code),
  grade TEXT NOT NULL CHECK (grade IN ('A','B','C','S')),
  PRIMARY KEY (account_id, position),
  UNIQUE (account_id, subject_code)
) STRICT, WITHOUT ROWID;
CREATE INDEX idx_student_subject_subject ON student_subject(subject_code);

CREATE TABLE student_achievement (
  achievement_id INTEGER PRIMARY KEY,
  account_id INTEGER NOT NULL REFERENCES account(account_id) ON DELETE CASCADE,
  position INTEGER NOT NULL CHECK (position BETWEEN 1 AND 30),
  activity TEXT NOT NULL CHECK (length(trim(activity)) BETWEEN 1 AND 80),
  kind TEXT NOT NULL CHECK (kind IN ('SPORT','COMPETITION','CLUB','ARTS','OTHER')),
  level TEXT NOT NULL CHECK (level IN
('SCHOOL','ZONAL','DISTRICT','PROVINCIAL','NATIONAL','INTERNATIONAL')),
  place TEXT NOT NULL CHECK (place IN ('FIRST','SECOND','THIRD','TAKING_PART')),
  year INTEGER NOT NULL CHECK (year BETWEEN 2000 AND 2100),
  UNIQUE (account_id, position)
) STRICT;

-- Mirrors shared/account.ts INTERESTS (pinned by worker/account.test.ts).
CREATE TABLE student_interest (
  account_id INTEGER NOT NULL REFERENCES account(account_id) ON DELETE CASCADE,
  interest TEXT NOT NULL CHECK (interest IN ('Building apps','Maths','Science and
labs','Working with people',
'Business and money','Teaching','Health and caring','Art and
design','Languages','Law and society',
'Nature and farming','Making and engineering','Sport')),
  PRIMARY KEY (account_id, interest)
) STRICT, WITHOUT ROWID;

-- Only student accounts carry student data (enforced here as well as in the API).
CREATE TRIGGER trg_student_profile_role BEFORE INSERT ON student_profile
WHEN (SELECT role FROM account WHERE account_id = NEW.account_id) IS NOT 'STUDENT'
BEGIN SELECT RAISE(ABORT, 'student_profile requires a STUDENT account'); END;
CREATE TRIGGER trg_student_achievement_role BEFORE INSERT ON student_achievement
WHEN (SELECT role FROM account WHERE account_id = NEW.account_id) IS NOT 'STUDENT'
BEGIN SELECT RAISE(ABORT, 'student_achievement requires a STUDENT account'); END;
CREATE TRIGGER trg_student_interest_role BEFORE INSERT ON student_interest
WHEN (SELECT role FROM account WHERE account_id = NEW.account_id) IS NOT 'STUDENT'
BEGIN SELECT RAISE(ABORT, 'student_interest requires a STUDENT account'); END;

```

Appendix D: Glossary

Table 13: Terms used in this document

Term	Meaning
BCNF	Boyce-Codd normal form: every determinant is a candidate key
Category (union type)	An EER subclass of the union of several superclasses
D1	Cloudflare's serverless SQL database, built on SQLite
Identifying relationship	The relationship that links a weak entity to the owner that identifies it
MVD	Multi-valued dependency, the basis of fourth normal form (4NF)
Read model	A query-optimised copy of data derived from the system of record
STRICT table	An SQLite table that rejects values of the wrong type
Weak entity	An entity that cannot be identified by its own attributes alone

Appendix E: References

Table 14: Sources referred to in this document

Ref	Source
[1]	R. Elmasri and S. Navathe, <i>Fundamentals of Database Systems</i> , 7th edition, Pearson, 2016 (chapters 3, 4, 9, 14, 15)
[2]	Cloudflare, <i>D1: Define foreign keys and SQL statements</i> , developers.cloudflare.com, retrieved 5 October 2026
[3]	Cloudflare, <i>D1 limits and pricing</i> , developers.cloudflare.com, retrieved 5 October 2026
[4]	SQLite, <i>STRICT Tables and Generated Columns</i> , sqlite.org
[5]	UGC handbook and cut-off table 2025/26 (see data/ugc/PROVENANCE.md)
[6]	ZP-DOC-03 Software Requirements Specification